

Name :- Rucha Nitin Pawar

Branch :- AI&DS

Div :- B

Roll No :- 70

Practical Batch :- B4

Aim:- Numerical computation with Numpy: perform different computation using NumPy library.

Theory

1. Explain the role of NumPy in python scientific computing ecosystem?

NumPy plays a central role in the Python scientific computing ecosystem by providing efficient tools for numerical operations on large datasets. It introduces the n-dimensional array (`ndarray`), which allows fast element-wise computations and supports vectorized operations, reducing the need for slow Python loops. NumPy also offers a wide range of mathematical functions, linear algebra routines, random number generation, and Fourier transforms, making it a foundation for many other libraries like pandas, SciPy, and scikit-learn. Its performance and flexibility make it essential for data analysis, scientific simulations, and machine learning.

2. Why is NumPy considered faster than pure python computations?

NumPy is considered faster than pure Python computations because it uses optimized C and Fortran libraries under the hood, allowing operations to be performed at a low level and in compiled code rather than in Python's slower interpreted loops. Its arrays (`ndarray`) are stored in contiguous memory blocks, which enables vectorized operations that apply computations to entire arrays at once instead of iterating element by element. This combination of memory efficiency, low-level optimizations, and vectorization makes NumPy much faster than standard Python lists and loops for numerical computations.

3. What types of mathematical operations can be performed using NumPy?

NumPy supports a wide range of mathematical operations, including:

1. Basic arithmetic operations – addition, subtraction, multiplication, division, and exponentiation on arrays.
2. Statistical operations – mean, median, standard deviation, variance, min, max, and sum.
3. Linear algebra operations – matrix multiplication, dot product, determinant, inverse, eigenvalues, and singular value decomposition.
4. Trigonometric functions – sine, cosine, tangent, arcsin, arccos, arctan, etc.
5. Exponential and logarithmic functions – `exp()`, `log()`, `log10()`.
6. Rounding and clipping – `round()`, `floor()`, `ceil()`, `clip()`.
7. Random number generation – generating random integers, floats, normal distributions, and permutation of arrays.

These operations can be performed efficiently on large datasets using NumPy's vectorized computations.

4. Explain indexing and slicing in NumPy arrays?

Indexing refers to selecting a specific element from an array using its position. For a one-dimensional array, you use a single integer index, e.g., `arr[2]` returns the third element. For multi-dimensional arrays, you provide indices for each dimension, e.g., `arr[1,3]` accesses the element in the second row and fourth column.

Slicing allows you to access a range of elements instead of a single value. You specify a start, stop, and optional step using the `start:stop:step` syntax. For example, `arr[1:5]` returns elements from index 1 to 4, and `arr[:,2]` returns every second element. In multi-dimensional arrays, slicing can be applied to each axis separately, e.g., `arr[0:2, 1:4]` extracts a subarray from the first two rows and columns 1 to 3.

5. How does NumPy integrate with libraries like: a.pandas, b.matplotlib, c.scikit-learn ?

NumPy integrates closely with many Python libraries by providing fast and efficient array structures and numerical operations:

- a. pandas – pandas Series and DataFrames are built on top of NumPy arrays, which allows fast vectorized operations, easy data manipulation, and efficient memory usage.
- b. matplotlib – NumPy arrays can be used directly as input for plotting functions, making it easy to visualize large datasets or mathematical functions efficiently.
- c. scikit-learn – scikit-learn uses NumPy arrays for input features and target labels, enabling efficient computations for machine learning algorithms, preprocessing, and model training.

```
import numpy as np
import time
```

```

# 1. Create array from list
print("1. Create NumPy Array:")
arr = np.array([1, 2, 3, 4, 5])
print(arr, "\n")

# 2. Basic arithmetic operations
print("2. Arithmetic Operations:")
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b, "\n")

# 3. Sum and Mean
print("3. Sum and Mean:")
arr2 = np.array([10, 20, 30, 40])
print("Sum:", np.sum(arr2))
print("Mean:", np.mean(arr2), "\n")

# 4. Reshape array
print("4. Reshape Array:")
arr3 = np.array([1, 2, 3, 4, 5, 6])
print(arr3.reshape(2, 3), "\n")

# 5. Indexing
print("5. Indexing:")
print("First element:", arr2[0])
print("Third element:", arr2[2], "\n")

# 6. Matrix addition and multiplication
print("6. Matrix Operations:")
m1 = np.array([[1, 2], [3, 4]])
m2 = np.array([[5, 6], [7, 8]])
print("Addition:\n", m1 + m2)
print("Multiplication:\n", np.dot(m1, m2), "\n")

# 7. Random numbers
print("7. Random Numbers:")
print("Random floats:", np.random.rand(3))
print("Random integers:", np.random.randint(1, 10, 5), "\n")

# 8. Standard deviation and variance
print("8. Standard Deviation & Variance:")
arr4 = np.array([1, 2, 3, 4, 5])
print("Std Dev:", np.std(arr4))
print("Variance:", np.var(arr4), "\n")

# 9. Mathematical functions
print("9. Mathematical Functions:")
arr5 = np.array([1, 4, 9, 16])
print("Square root:", np.sqrt(arr5))
print("Log:", np.log(arr5))
print("Exponential:", np.exp(arr5), "\n")

# 10. Python list vs NumPy array
print("10. Performance Comparison:")
list1 = list(range(1000000))
array1 = np.array(list1)

start = time.time()
result = [x * 2 for x in list1]
print("List Time:", time.time() - start)

start = time.time()
result = array1 * 2
print("NumPy Time:", time.time() - start, "\n")

# 11. Vectorized operations
print("11. Vectorized Operation:")
arr6 = np.array([1, 2, 3, 4])
print(arr6 * 2, "\n")

# 12. Solve linear equations
print("12. Linear Equation Solution:")
A = np.array([[2, 1], [1, 3]])

```

```

B = np.array([8, 13])
solution = np.linalg.solve(A, B)
print("Solution:", solution, "\n")

# 13. Broadcasting
print("13. Broadcasting:")
arr7 = np.array([1, 2, 3])
print(arr7 + 2, "\n")

# 14. Large dataset analysis
print("14. Large Dataset Analysis:")
data = np.random.rand(1000)
print("Mean:", np.mean(data))
print("Max:", np.max(data))
print("Min:", np.min(data), "\n")

# 15. Optimized computation
print("15. Optimized Computation:")
arr8 = np.arange(1000000)
result = np.square(arr8)
print("Computation Done Successfully!")

```

```

1. Create NumPy Array:
[1 2 3 4 5]

2. Arithmetic Operations:
Addition: [5 7 9]
Subtraction: [-3 -3 -3]
Multiplication: [ 4 10 18]
Division: [0.25 0.4 0.5 ]

3. Sum and Mean:
Sum: 100
Mean: 25.0

4. Reshape Array:
[[1 2 3]
 [4 5 6]]

5. Indexing:
First element: 10
Third element: 30

6. Matrix Operations:
Addition:
[[ 6  8]
 [10 12]]
Multiplication:
[[19 22]
 [43 50]]

7. Random Numbers:
Random floats: [0.66792625 0.4332188 0.04426101]
Random integers: [4 1 6 3 4]

8. Standard Deviation & Variance:
Std Dev: 1.4142135623730951
Variance: 2.0

9. Mathematical Functions:
Square root: [1. 2. 3. 4.]
Log: [0. 1.38629436 2.19722458 2.77258872]
Exponential: [2.71828183e+00 5.45981500e+01 8.10308393e+03 8.88611052e+06]

10. Performance Comparison:
List Time: 0.05527973175048828
NumPy Time: 0.017855405807495117

11. Vectorized Operation:
[2 4 6 8]

12. Linear Equation Solution:
Solution: [2.2 3.6]

```

13. Broadcasting:

[3 4 5]

14. Large Dataset Analysis:

Mean: 0.5061585496138673

Max: 0.9999785869251901

Conclusion:

This practical demonstrated how to use the NumPy library for numerical computations in Python. It helped in understanding array creation, mathematical operations, and efficient data handling. NumPy makes calculations faster and easier compared to traditional methods, making it very useful for scientific and data analysis tasks.